



FreshFind — *Fresh All Along.*

Complete 13-Page Documentation — a local farmers-market discovery platform with live opening status, seasonal produce guides, geolocation proximity search, bookmarks, a rule-based chatbot and a cinematic editorial UI theme.

React 19 + TypeScript

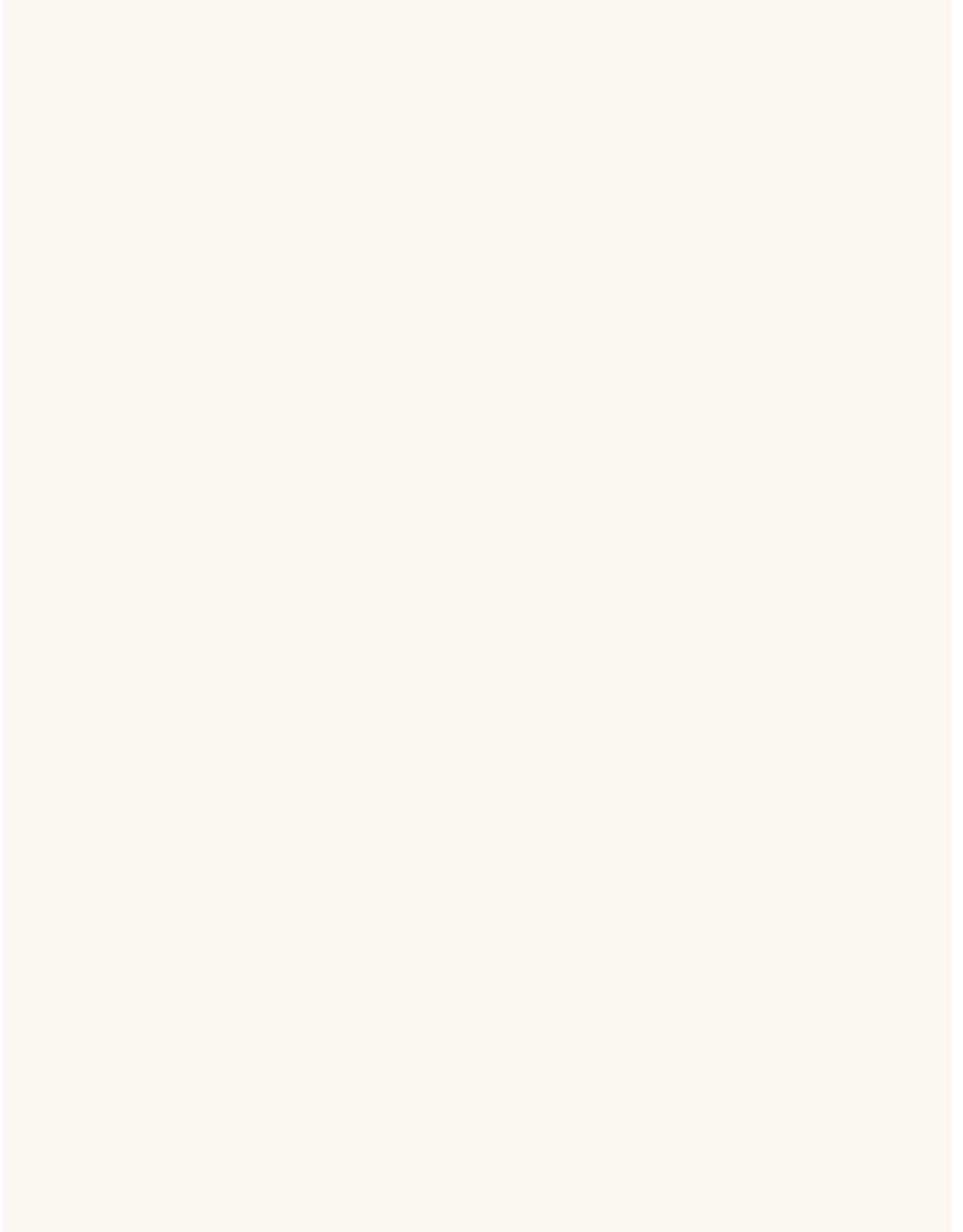
Tailwind CSS 4

Vite 7

Zero Backend

Single-File Build

Scroll down to explore — or jump to a page: [Tech Stack](#) · [Design System](#) · [Pages](#) · [Components](#) · [Deployment](#)



1. Introduction & Product Overview

FreshFind is a single-page application (SPA) that maps local farmers markets, seasonal produce and the growers behind the stalls — so shoppers can answer three questions in seconds: *what is in season, where can I buy it, and is it open right now?*

1.1 The Problem It Solves

Market information is scattered across noticeboards, social posts and stale listings. FreshFind consolidates it into one cinematic, fast, mobile-friendly experience: twelve markets, sixteen produce guides, six growers — all mapped, all checked, all local.

1.2 Core Value Propositions



Live Status

Open / Opening-Soon / Closed is recomputed every 30 seconds from the device clock — never cached on a server.



Proximity Search

One tap sorts every market by haversine distance from the browser's geolocation. Nothing is uploaded.



Season-First Data

Produce is listed only in its natural window; the current season is derived from the real calendar.



Local Chatbot

A rule-based assistant answers questions about markets, timings, seasons and produce with deep links.

1.3 Feature Summary

MODULE	HIGHLIGHTS
Home	Auto-rotating cinematic hero, marquee ticker, Quick Find, 3D rotating market wheel, tabbed categories, featured slider, editorial video band, growers & testimonial sliders, animated visitor counter
Find a Market	Geolocation sort, synced map-plot + list view, live status pills, keyword/day/produce filters
Market Directory	Filter sidebar (area, day, produce, keyword, open-now), 4 sort modes, grid / list / map views, skeleton loading
Market Detail	Weekly schedule with “today” highlight, stocked produce, seasonal panel, embedded map, nearby markets, save & share
Produce Guide	8 category tabs, 16 guides, per-item market counts, availability strip
Produce Detail	Buy/store/cook tips, stocking markets, pairing suggestions, scroll-driven 3D artwork, save & share
Seasonal	Four season tabs, parallax feature panel, “where to buy this season”, year overview strip
My Fresh Finds	localStorage bookmarks, personal notes, tabbed filter, export to .txt, share links

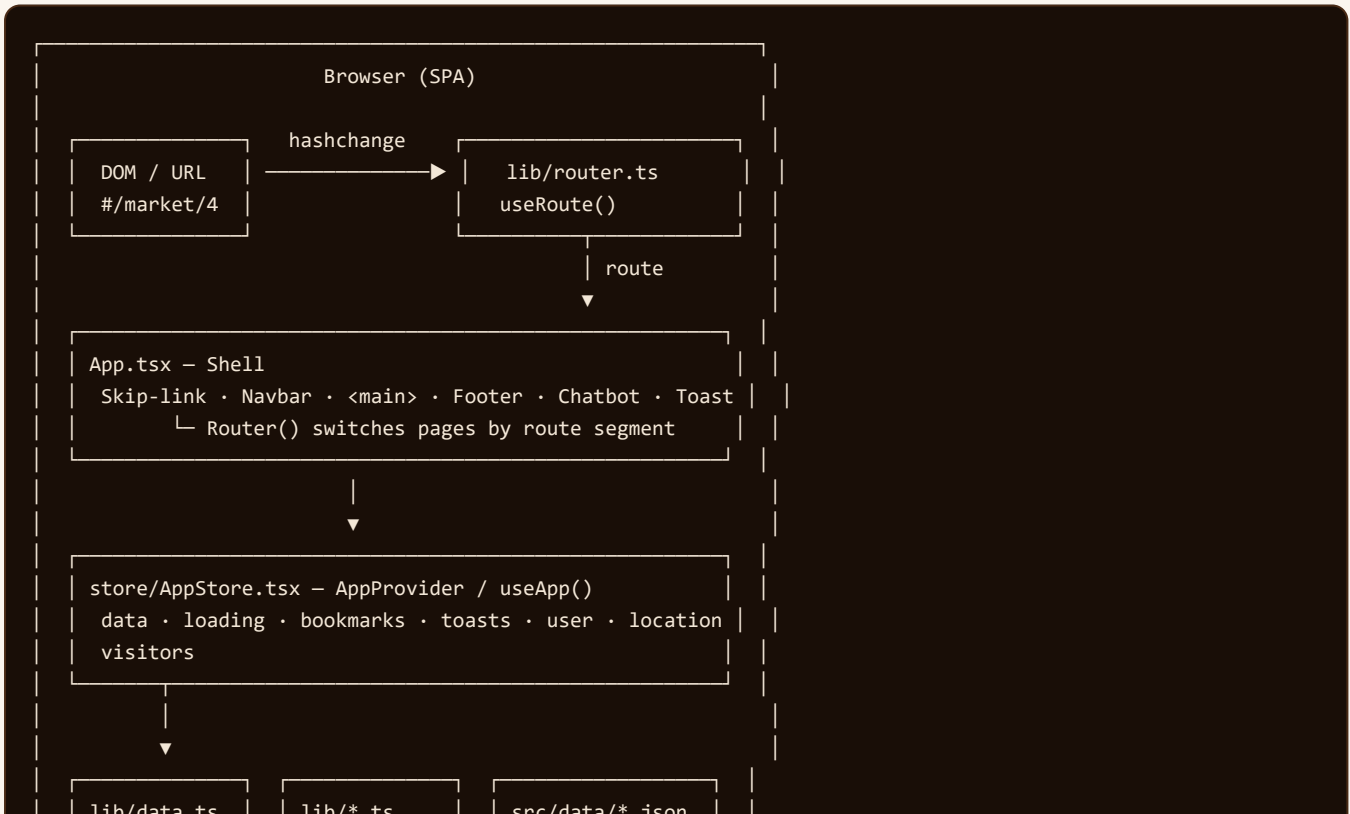
2. Technology Stack & Architecture

A deliberately lean, modern front-end stack. Everything runs in the browser; the production build inlines all JS/CSS into one HTML file.

2.1 Stack at a Glance

LAYER	TECHNOLOGY	VERSION	ROLE
UI library	React	19.2	Component model, hooks, context
Language	TypeScript	5.9	Typed contracts for the whole data layer
Build tool	Vite	7.3	Dev server, HMR, bundling
Styling	Tailwind CSS	4.1	Utility-first, <code>@theme</code> design tokens
CSS compile	Lightning CSS (via Vite)	—	Fast transforms, minification
Routing	Custom hash router	—	Zero-dependency SPA navigation
State	React Context + hooks	—	Global app store, no external lib
Storage	Web Storage API	—	Bookmarks, session, visitor counter
Maps	Google Maps embed	—	Keyless <code>output=embed</code> iframe
Artwork	Inline SVG + PNG cut-outs	—	Vector-first with graceful fallback

2.2 Runtime Architecture



3. Project Structure & File Organization

A flat, convention-driven layout: pages present, components compose, `lib` holds logic, `data` holds content.

3.1 Directory Tree

```
freshfind/
├─ index.html           # Vite entry (root div + font links)
├─ package.json
├─ vite.config.ts       # plugins + @ alias + singlefile
├─ tsconfig.json
├─ public/
│  └─ images/           # logo.png + 22 produce cut-out PNGs
│     ├── logo.png
│     ├── avocado-cut.png tomato-cut.png strawberry-cut.png
│     └── carrot-cut.png broccoli-cut.png basil-cut.png ...
├─ scripts/
│  └─ cutout.mjs        # asset pipeline (background removal)
└─ src/
   ├── main.tsx         # React root + StrictMode
   ├── index.css        # design system: @theme, keyframes, utilities
   ├── App.tsx          # shell + hash router switch
   ├── pages/           # 10 route targets
   │  ├── Home.tsx FindMarket.tsx Directory.tsx MarketDetail.tsx
   │  ├── ProduceGuide.tsx ProduceDetail.tsx Seasonal.tsx
   │  └── MyFinds.tsx About.tsx Contact.tsx NotFound.tsx
   ├── components/
   │  ├── Navbar.tsx Footer.tsx Hero.tsx QuickFind.tsx
   │  ├── Sections.tsx cards.tsx MapPanel.tsx Chatbot.tsx
   │  └── ProduceArt.tsx Doodles.tsx Logo.tsx ui.tsx
   ├── store/
   │  └─ AppStore.tsx   # context provider (global state)
   └─ lib/
      ├── data.ts types.ts filters.ts marketStatus.ts
      ├── hooks.ts router.ts utils.ts
      └─ data/          # ALL content as typed JSON
         ├── markets.json # 12 markets
         ├── produce.json # 16 produce guides
         ├── seasonal.json # 4 seasons
         ├── farmers.json # 6 growers
         ├── testimonials.json # 5 testimonials
         └─ chatbot.json  # intents + replies
```

3.2 Separation of Concerns

FOLDER	RESPONSIBILITY	NEVER CONTAINS
<code>pages/</code>	Route-level composition, page state	Reusable widgets
<code>components/</code>	Reusable UI (presentational + smart)	Raw JSON content
<code>lib/</code>	Pure logic: filters, status engine, hooks	JSX
<code>data/</code>	Typed content JSON — the “database”	Logic, JSX
<code>store/</code>	Global state + persistence side-effects	UI markup

4. Design System & UI Theme

The visual language is “*premium organic / editorial / cinematic*” — a dark forest-brown canvas, warm cream content bands, editorial serif display type and playful hand-drawn produce art.

4.1 Color Palette



Two-tone rhythm: dark sections (heroes, feature bands, footer) alternate with cream sections so the page breathes.

`color-scheme: dark` is set globally; light sections override heading colors locally.

4.2 Typography

Fraunces (display) Fresh All Along — headings

Manrope (sans) Body copy, labels, UI

Parisienne (script) *italic accent words inside headings*

Scale is fluid with `clamp()` — e.g. hero headings run `clamp(3rem, 10vw, 7.5rem)`. Display headings use tight `leading-[0.98]` and `letter-spacing: -0.02em` for an editorial feel.

4.3 Signature UI Elements

ELEMENT	DESCRIPTION
Organic curve dividers	<code><Curve></code> — SVG waves that flow between dark and cream sections
Squiggle underline	Hand-drawn SVG stroke under script accent words (<code>.squiggle</code>)
Glass navbar	Frosted dark-brown bar with a wavy bottom edge and thin red hairline
Status pills	Open Now (green pulse) / Opening Soon (citrus) / Closed (muted)
Floating produce art	SVG/PNG cut-outs with 3D drop shadows and perspective tilt animations
3D market wheel	CSS <code>preserve-3d</code> carousel of cards that pauses on hover
Doodle scatter	Sparkles, clovers and sprouts floating over cream sections
Grain overlay	Inline SVG noise at 4.5% opacity for a printed-editorial texture
Marquee ticker	“Fresh today” strip with edge-fade mask, 34 s loop

4.4 Elevation & Motion Tokens

- Shadows:** `--shadow-soft` (cards), `--shadow-lift` (hover), `--shadow-glow` (red accent).

5. Routing & Navigation System

A 58-line hash router gives the SPA real URLs, history support and shareable deep links with zero dependencies.

5.1 Route Table

HASH URL	PAGE	NOTES
#/	Home	Full marketing landing
#/find	Find a Market	Geolocation + map/list
#/directory	Market Directory	Filters, sorts, views
#/market/:id	Market Detail	12 markets, else 404
#/produce	Produce Guide	Category tabs
#/produce/:id	Produce Detail	16 guides, else 404
#/seasonal	Seasonal Picks	4-season explorer
#/saved	My Fresh Finds	Bookmarks + notes
#/about	About	Story, values, team
#/contact	Contact	Form + map
anything else	NotFound	Styled 404

Query strings are supported too — the Directory pre-fills filters from `?area=&day=&produce=`, enabling shareable filtered views.

5.2 Router Implementation

```
// lib/router.ts – the whole router
function parse(hash) {
  const clean = hash.replace(/^#/,"") || "/";
  const [pathPart, queryPart] = clean.split("?");
  return { path, segments: path.split("/").filter(Boolean), query };
}
export function useRoute() { // re-renders on hashchange
  const [route, setRoute] = useState(() => parse(window.location.hash));
  useEffect(() => {
    const onChange = () => setRoute(parse(window.location.hash));
    window.addEventListener("hashchange", onChange);
    return () => window.removeEventListener("hashchange", onChange);
  }, []);
  return route;
}
```

6. Pages I — Home, Find a Market, Directory

6.1 Home — the Cinematic Landing

The Home page is a composed scroll-experience built from reusable sections:

#	SECTION	WHAT IT DOES
01	Hero	5 auto-rotating backdrop photos (Ken-Burns drift), floating produce layer, masked line-by-line heading, live clock, slide dots
02	Ticker	Infinite marquee of “fresh today” items
02.5	Quick Find	Compact filter panel + result cards
03	Market Wheel	3D rotating carousel of open-now markets
04	Choose Your Find	6 category tabs with tinted panels
05	Featured Produce	Autoplaying slider with save buttons
06	Avocado Feature	4-point checklist, scroll-scaled 3D art
07	Fresh Picks	6 produce cards with hearts
08	Seasonal Teaser	4 season cards, emoji produce lists
09	Fresh All Along	Full-bleed parallax editorial band
10	Made With Love	Zoom background + YouTube lightbox modal
11	Growers	Autoplay slider of 6 farmer cards
12	Testimonials	Slider of 5 quotes with star ratings
13	Counter Strip	Animated count-up visitor counter

6.2 Find a Market — Proximity Search

- One-tap **Use My Location** → browser geolocation → all 12 markets sorted by haversine distance.
- Synced **map plot + list**: selecting a market highlights it on both; hovering pins shows name + distance tooltip.
- Fallback: if geolocation is denied, sorting uses the city centre coordinates and a toast explains.
- Live status pills next to every market row; “Locating...” busy state.

6.3 Market Directory — the Workhorse

- **Filter sidebar** (sticky on desktop, collapsible on mobile): keyword with datalist, area chips, day-of-week grid, produce-type select, “Open now only” toggle.
- **Sort modes**: next open day (default), alphabetical, proximity (auto-requests location), top rated.
- **Views**: grid / list / map — state preserved while filtering.
- **Status**: skeleton cards while data loads, friendly empty state with reset action, result count + open now count

7. Pages II — Details, Seasons, Saved, About, Contact

7.1 Market Detail

- **Hero:** full-bleed photo, area chip, live status pill, rating & stall count, description, *Save + Share* actions, live clock.
- **About:** established year, address, features checklist chips.
- **Weekly schedule:** 7-row table with today highlighted (“Today” badge, tinted row) and closed days marked.
- **What you'll find:** produce cards linking to produce details.
- **Seasonal panel:** tinted card for the current season with overlapping produce art.
- **Location:** keyless Google Maps embed with illustrated fallback and “Directions” link.
- **Sidebar:** sticky opening-status card with facts (hours, days, stalls, distance) and a CTA to Find a Market.
- **Nearby markets:** 3 closest by straight-line distance.

7.2 Produce Guide

- Editorial header with oversized floating produce art (avocado, tomato, lemon, basil) drifting on scroll.
- 8 category filter pills with item counts; grid of produce cards.
- Stats band (16 guides / 12 markets / 4 seasons) and a “where everything is stocked” quick list.

7.3 Produce Detail

- **Hero:** item photo with color-tinted radial glow, category label, tagline in display italic, oversized 3D artwork that scales with scroll progress (`useScrollProgress`).
- **Facts sidebar:** category, season, peak, origin, market count, pairing chips.
- **Tips:** numbered buy/store/cook cards.
- **Markets:** cards of every market stocking the item, each with a live status pill (“open now” mini-list in sidebar).
- **Goes well with:** related produce cards.

7.4 Seasonal

- Four season tabs (Spring–Winter) with month ranges; current season auto-selected and badged “Now”.
- Feature panel: parallax image with gradient blend, season note, produce list with peak times.
- “Where to find it” — markets stocking the season's produce, with status pills.
- “The whole year” strip — dark panel with all four seasons.

7.5 My Fresh Finds (Bookmarks)

- Saved markets & produce with thumbnails, subtitles and live status.
- **Personal notes:** per-item textarea, saved on blur to localStorage.

8. Data Layer & Content Model

All content lives in typed JSON under `src/data/` — components never hardcode market or produce copy.

8.1 Content Inventory

FILE	RECORDS	KEY FIELDS
<code>markets.json</code>	12	name, area, address, days[], hours, produce[], categories[], lat/lng, rating, stalls, established
<code>produce.json</code>	16	name, category, tagline, description, season[], peak, markets[], color, origin, tips[], pairs[]
<code>seasonal.json</code>	4	name, months, note, accent, tint, image, items[]
<code>farmers.json</code>	6	name, farm, market, specialty, years, quote, tint, image
<code>testimonials.json</code>	5	name, role, location, initials, tint, rating, quote
<code>chatbot.json</code>	intents[]	keywords[], response, bullets[], links[], suggestions[]

8.2 TypeScript Contracts

```
// lib/types.ts — every JSON shape has a matching interface
export interface Market {
  id: number; name: string; area: string; city: string; address: string;
  days: string[]; openingTime: string; closingTime: string;
  produce: string[]; categories: string[]; image: string;
  lat: number; lng: number; rating: number; stalls: number;
  established: number; description: string; features: string[];
}
export interface ProduceItem { id: string; name: string; category: string;
  tagline: string; description: string; season: string[]; peak: string;
  markets: number[]; image: string; color: string; origin: string;
  tips: string[]; pairs: string[]; }
export interface Bookmark { key: string; type: "market" | "produce";
  id: string; name: string; subtitle: string; image: string;
  note: string; addedAt: number; }
```

8.3 Derived Selectors

- `marketsForProduce(id)` — which markets stock an item.
- `produceForMarket(id)` — which items a market stocks.
- `currentSeasonId(date)` — month → spring/summer/autumn/winter.
- `AREAS` — sorted unique areas; `PRODUCE_CATEGORIES` — fixed category order.

8.4 The Market Status Engine

```
// lib/marketStatus.ts — pure functions, device clock is the only input
getMarketStatus(market, now) → {
  status: "open" | "opening-soon" | "closed",
  label, message, // "Open for 2h 15m" · "Opens in 34 min"
```

9. State Management & Browser Storage

One React Context (`AppStore`) holds all global state; `localStorage` provides persistence across sessions.

9.1 Store Shape

STATE	TYPE	PERSISTENCE	USED FOR
<code>data</code>	<code>AppData null</code>	—	Loaded JSON content
<code>loading</code>	<code>boolean</code>	—	Skeleton states
<code>bookmarks</code>	<code>Bookmark[]</code>	✓ <code>freshfind.bookmarks.v1</code>	My Fresh Finds
<code>toasts</code>	<code>Toast[]</code>	—	Ephemeral notifications (3.2 s)
<code>user</code>	<code>{name, email} null</code>	✓ <code>freshfind.user.v1</code>	Demo auth session
<code>loginOpen</code>	<code>boolean</code>	—	Login modal visibility
<code>location</code>	<code>Coords null</code>	—	Geolocation (memory only)
<code>visitors</code>	<code>number</code>	✓ <code>freshfind.visitor.count</code>	Animated counter

9.2 Key Interactions

- **Save / unsave:** `toggle()` adds or removes a bookmark (keyed `market:4` / `produce:avocado`) and fires a toast — one function powers every heart button in the app.
- **Notes:** `setNote()` patches a single bookmark's note on blur; persisted automatically by the bookmark effect.
- **Auth:** `signIn(name, email)` stores the session locally (demo only — no password verification, no network); the navbar swaps to a signed-in chip with sign-out.
- **Geolocation:** `locate()` requests the position, toasts the outcome, and falls back to city-centre coordinates so proximity sort still works.
- **Visitor counter:** a deterministic per-browser base (12 482 + random 0–639) incremented once per app load.

9.3 Storage Safety

```
// every write is guarded – private mode / disabled storage won't crash
useEffect(() => {
  try { localStorage.setItem(BOOKMARK_KEY, JSON.stringify(bookmarks)); }
  catch { /* storage unavailable – state lives in memory for the session */ }
}, [bookmarks]);
```

9.4 Toast System

Toasts are tone-tagged (`success` / `neutral` / `warn`), auto-dismiss after 3.2 seconds, stack bottom-left, and are announced via `aria-live="polite"` for screen readers.

10. Component Library

A small, well-factored kit — most UI is built from ~30 components across 14 files.

10.1 Primitives (`components/ui.tsx`)

COMPONENT	PURPOSE
<code>Btn</code> / <code>BtnLink</code>	6 variants × 3 sizes, sweep highlight, optional arrow — buttons and hash links share one style
<code>Reveal</code>	Declarative scroll-reveal wrapper (7 variants, configurable delay, polymorphic <code>as</code>)
<code>MaskedHeading</code>	Line-by-line typographic reveal with clip boxes for tall script type
<code>SectionLabel</code>	Editorial “01 — DISCOVER” markers (dark/light tones)
<code>Curve</code>	Organic SVG wave divider (flippable, custom path)
<code>StatusPill</code>	Open / Opening Soon / Closed badge with pulse glow
<code>LiveClock</code>	1 Hz digital clock with pulsing dot
<code>Modal</code>	Accessible dialog: Escape to close, focus management, scroll-lock friendly
<code>Slider</code>	Drag + arrows + snap carousel with optional autoplay (pauses on hover/focus)
<code>Breadcrumbs</code> · <code>ToastHost</code> · <code>HeartButton</code> · <code>Skeleton</code> · <code>VisitorCounter</code>	Navigation, feedback, bookmarking, loading, count-up

10.2 Cards (`components/cards.tsx`)

COMPONENT	VARIANTS / NOTES
<code>MarketCard</code>	grid + list variants; accent rotation, save heart, status pill, distance badge
<code>ProduceCard</code>	cream/white/green tones; oversized floating art; season + market count
<code>FarmerCard</code>	Photo card with specialty chip, years, market link, quote
<code>TestimonialCard</code>	5-star row, quote, initials avatar

10.3 Feature Components

COMPONENT	FILE	HIGHLIGHT
Navbar + LoginModal	Navbar.tsx	Glass wave, mobile menu, demo auth
Hero + Ticker	Hero.tsx	5-slide Ken-Burns carousel, floating produce
QuickFind + MarketWheel + OpenNowRail	QuickFind.tsx	Filters, 3D carousel, open-now cards

11. Animations & Interaction Systems

Motion is organic, purposeful and fully opt-out for users who prefer reduced movement.

11.1 Scroll Reveal System

One shared `IntersectionObserver` adds `is-visible` to any `.reveal` element. Seven variants: `up`, `down`, `left`, `right`, `zoom`, `blur` and `mask` — all on the organic easing curve.

11.2 Keyframe Library (`ff-*`)

ANIMATION	DURATION	USED FOR
<code>ff-float</code> / <code>ff-float-slow</code>	7 s / 11 s	3D perspective produce drift
<code>ff-pop3d</code>	1 s	Tilt-in reveal of cut-outs
<code>ff-marquee</code>	34 s	Ticker + skeleton shimmer
<code>ff-pulse-ring</code>	2.4 s	Status dots, chat launcher, play button
<code>ff-pop</code>	0.5 s	Chat bubbles, panel transitions
<code>ff-spin</code>	22 s	Slow decorative rotation
<code>ff-scroll-dot</code>	2 s	Hero scroll indicator
<code>ff-kenburns</code>	9 s	Hero backdrop drift per slide
<code>ff-wheel-spin</code>	32 s	3D market wheel (pauses on hover)

11.3 Scroll-Driven Effects

- **Parallax** (`useParallax`) — rAF-throttled translate on hero images and story layers.
- **Scroll progress** (`useScrollProgress`) — writes a `--p` custom property (0→1) that scales/rotates the avocado feature art, the produce detail artwork and the video zoom band.
- **Count-up** (`UseCountUp`) — eased visitor counter triggered when scrolled into view.

11.4 Reduced Motion

```
@media (prefers-reduced-motion: reduce) {
  * { animation-duration: 0.001ms !important;
    transition-duration: 0.001ms !important; }
  .reveal { opacity: 1 !important; transform: none !important; }
}
```

Hooks (`useReducedMotion`, `useCountUp`, parallax) also short-circuit, so OS-level preferences are respected everywhere.

12. Accessibility, Performance & Browser Support

12.1 Accessibility Checklist

AREA	IMPLEMENTATION
Landmarks	<code>header</code> / <code>nav</code> / <code>main</code> / <code>footer</code> on every page
Skip link	“Skip to content” — visible on focus, jumps to <code>#main</code>
Focus	<code>:focus-visible</code> ring in accent red, 3 px offset
Widgets	Tabs use <code>role="tablist"</code> + <code>aria-selected</code> ; chat panel is <code>role="dialog"</code> with <code>inert</code> when closed
Live regions	Toasts use <code>aria-live="polite"</code> ; status text is <code>sr-only</code> where decorative
Labels	Every input has a label (visible or <code>sr-only</code>); icons carry <code>aria-label</code>
Motion	Full <code>prefers-reduced-motion</code> support (see 11.4)

12.2 Performance Choices

- **Vector-first artwork** — inline SVGs scale perfectly with zero network cost; PNG cut-outs only where supplied.
- **Lazy images** — `loading="lazy"` everywhere except the first hero slide (`fetchpriority="high"`).
- **Single observer** — one IntersectionObserver shared by all reveal elements instead of one per element.
- **Throttled scroll** — parallax/scroll effects run through `requestAnimationFrame` with passive listeners.
- **CSS-only motion** — transforms and opacity only (compositor-friendly), `will-change` hints on animated art.
- **Zero runtime deps** beyond React — no router lib, no state lib, no CSS framework JS.

12.3 Browser Support

Current evergreen browsers: Chrome / Edge, Firefox, Safari (desktop & iOS). Requires support for IntersectionObserver, CSS custom properties, `clamp()`, `backdrop-filter` (graceful degradation) and ES2020. Google Maps and YouTube embeds degrade to illustrated fallbacks.

13. Deployment, Testing & Documentation Map

13.1 Running the Project

```
npm install      # install dependencies
npm run dev      # dev server → http://localhost:5173
npm run build    # production build → dist/
npm run preview  # preview the production build
```

13.2 Deployment

Because `vite-plugin-singlefile` inlines everything, the whole site is one self-contained `dist/index.html`:

- **Static hosts:** drag-and-drop to Netlify / Vercel / GitHub Pages / Cloudflare Pages — no server config needed.
- **Any web server:** copy the single file behind nginx, Apache or S3 + CloudFront.
- **Offline:** the file works from a USB stick or local filesystem (Google Maps / YouTube features need network).

13.3 Quality Strategy

PRACTICE	WHERE
Type safety	Strict TypeScript contracts for all JSON data
Pure logic	Filters, status engine, selectors — easily unit-testable
Defensive storage	Guarded localStorage writes, safe JSON parsing
Graceful fallbacks	Map illustration, SVG art, city-centre location
Loading & empty states	Skeletons in Directory, empty states with CTAs

13.4 Documentation Map

This document is organized as 13 readable pages:

- 01 Introduction & Product Overview
- 02 Technology Stack & Architecture
- 03 Project Structure & File Organization
- 04 Design System & UI Theme
- 05 Routing & Navigation System
- 06 Pages I — Home, Find a Market, Directory
- 07 Pages II — Details, Seasons, Saved, About, Contact
- 08 Data Layer & Content Model
- 09 State Management & Browser Storage
- 10 Component Library
- 11 Animations & Interaction Systems